

1. 본인확인-표준창 암호화용 키 정보 파일에서 정보 가져오기

1 본인확인 암호화용 키 정보 (mok_keyInfo.dat) 파일에서 정보 가져오기

⚠ 중요

키파일은 서버 내 안전한 장소에 보관

서비스 등록 후 발급된 mok_keyInfo.dat 를 안전한 서버 내 디렉토리에 보관

Step 1. 암호화된 키 파일 준비

- **Step 1-1.** 서비스 등록 후 발급받은 본인확인 암호화 키 정보(예 : mok_keyInfo.dat)파일을 준비합니다.
- **Step 1-2.** 암호화 키 정보 파일을 byte[] 배열로 읽어서 준비합니다.

❗ 키 파일은 본인확인 인증결과 데이터를 복호화하기 위한 암호화된 정보를 포함합니다.

Step 2. AES 복호화 키 및 IV 생성

- **Step 2-1.** 암/복호화를 위한 해시함수에 SHA-256 해시 인스턴스를 생성합니다.
- **Step 2-2.** mobileOK_password(키 파일 비밀번호)를 해싱합니다.
 - Hash1(AES_KeyBytes) : mobileOK_password(키 파일 비밀번호)의 byte를 가지고 해싱합니다.
 - 해싱 결과의 앞 16바이트를 AES_KeyBytes (32byte의 앞16byte)로 사용합니다.
- **Step 2-3.** 해싱을 한 번 더 실행하여 IV(초기화 벡터) 및 AES 키의 나머지 부분을 생성합니다.
 - Hash2 : 해싱 결과(Hash1)을 한번 더 해싱합니다.
 - AES_KeyBytes (Hash1) : Hash1의 앞 16byte + Hash2의 뒤 16byte
 - AES_IvBytes (초기화 벡터 16byte) : Hash2의 앞 16byte

❗ 이 과정은 mok_keyInfo.dat 파일을 복호화하기 위해 AES 256bit 키와 16byte IV를 생성하는 단계입니다. 키 파생에는 SHA-256 해시를 2회 적용하여 키와 IV를 분리 생성합니다.

- AES_KeyBytes: 32바이트 AES 키 (Hash1 앞 16바이트 + Hash2 뒤 16바이트).
- AES_IvBytes: 16바이트 초기화 벡터 (Hash2 앞 16바이트).

Step 3. 데이터 복호화

- **Step 3-1.** AES/CBC/PKCS5Padding 모드로 위에서 생성한 AES 키와 IV를 이용하여 데이터를 복호화합니다.

Step 4. 결과 확인

- **Step 4-1.** 복호화된 결과를 JSON, UTF-8 문자열로 변환

⚠ 중요

JSON 형태로 복호화가 되는지 검증 필수

```
{
  "ServiceId" : "7a2d276f-2bd0-4b60-852b-b91da40dd9d3",
  "ClientPrivateKey" : "MIIEvAIBADANBgkqhkiG9w0BAQE ... TcWg==",
  "ServerPublicKey" : "MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ ... 8DAQAB"
}
```

📘 Info

ServiceId : 이용기관 키파일의 고유 ID (인증 요청 시 서비스 식별자로 활용됨)

ClientPrivateKey : 개인정보 복호화 시 사용됨 (→ [5. 본인확인-표준창 개인정보 복호화 참고](#))

ServerPublicKey : 거래요청 시 클라이언트 정보 암호화에 사용됨 (→ [2. 본인확인-표준창 거래요청정보 생성 참고](#))

샘플은 JAVA로 제공됩니다.

```

public class MokKeyInfoDecryption {

    public static String decryptMokKeyInfo(String filePath, String mobileOKPassword) throws Exception {
        // Step 1: 키 파일 준비
        // Step 1-1 & 1-2: mok_keyInfo.dat 파일을 byte[]로 읽기
        byte[] encryptedData = Files.readAllBytes(Paths.get(filePath));

        // Step 2: AES 복호화 키 및 IV 생성
        // Step 2-1: SHA-256 해시 인스턴스 생성
        MessageDigest sha256 = MessageDigest.getInstance("SHA-256");

        // Step 2-2: mobileOK_password 해싱 (Hash1)
        byte[] passwordBytes = mobileOKPassword.getBytes(StandardCharsets.UTF_8);
        byte[] hash1 = sha256.digest(passwordBytes); // 32바이트 해시 결과
        byte[] aesKeyBytes = new byte[32]; // AES 키를 위한 32바이트 배열 준비
        System.arraycopy(hash1, 0, aesKeyBytes, 0, 16); // Hash1의 앞 16바이트를 AES 키 앞부분으로 사용

        // Step 2-3: 두 번째 해싱 (Hash2)으로 IV와 AES 키 나머지 부분 생성
        byte[] hash2 = sha256.digest(hash1); // Hash1을 다시 해싱
        System.arraycopy(hash2, 16, aesKeyBytes, 16, 16); // Hash2의 뒤 16바이트를 AES 키 뒷부분으로 사용
        byte[] aesIvBytes = Arrays.copyOfRange(hash2, 0, 16); // Hash2의 앞 16바이트를 IV로 사용

        // Step 3: 데이터 복호화
        // Step 3-1: AES/CBC/PKCS5Padding 모드로 복호화
        Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        SecretKeySpec secretKeySpec = new SecretKeySpec(aesKeyBytes, "AES");
        IvParameterSpec ivSpec = new IvParameterSpec(aesIvBytes);
        aesCipher.init(Cipher.DECRYPT_MODE, secretKeySpec, ivSpec);

        byte[] decryptedData = aesCipher.doFinal(encryptedData);

        // Step 4: 결과 확인
        // Step 4-1: 복호화된 결과를 UTF-8 JSON 문자열로 변환
        String jsonResult = new String(decryptedData, StandardCharsets.UTF_8);

        // JSON 검증 (간단히 출력으로 확인, 실제로는 JSON 파싱 후 검증 필요)
        System.out.println("Decrypted JSON: " + jsonResult);
        return jsonResult;
    }
}

```

```
}

// 테스트용 메인 메서드
public static void main(String[] args) throws Exception {
    String filePath = "path/to/mok_keyInfo.dat"; // 실제 파일 경로로 변경
    String mobileOKPassword = "your_mobileOK_password"; // 실제 비밀번호로 변경

    String decryptedResult = decryptMokKeyInfo(filePath, mobileOKPassword);

    // JSON 형태로 복호화되었는지 검증 (예: ServiceId, ClientPrivateKey, ServerPublicKey 포함 여부 확인)
    ObjectMapper mapper = new ObjectMapper();
    Map<String, String> result = mapper.readValue(decryptedResult, Map.class);
    if (result.containsKey("ServiceId") && result.containsKey("ClientPrivateKey") && result.containsKey("ServerPublicKey")) {
        System.out.println("JSON 복호화 성공");
    } else {
        System.out.println("JSON 복호화 실패: 올바른 JSON 형식이 아님");
    }
}
}
```

2. 본인확인-표준창 거래요청정보 생성

2 본인확인 거래 요청 정보 생성

Summary

clientTxId 는 본인확인 요청과 결과를 매핑하고 재사용을 방지하기 위한 **고유 식별자**입니다.
이 값은 요청과 응답의 일치성을 검증하고, 인증 결과 재사용을 방지하는 데 사용됩니다.

Step 1. 이용기관 거래 ID (clientTxId) 생성

- **Step 1-1.** 중복되지 않는 이용기관의 거래ID (clientTxId)를 생성합니다.

이용기관 거래 ID 생성 규칙

- 거래ID는 최소 20자 이상 40자 이내로 생성
- 본인확인 이용기관 거래ID 는 유일한 값이어야 하며 기 사용한 거래ID가 있는 경우 오류 발생
- 이용기관이 고유식별 ID로 유일성을 보장할 경우 고객이 이용하는 ID사용 가능

예시) [이용기관 식별자] + [UUID]

```
"ORG001-550e8400-e29b-41d4-a716-4466554"
```

인증 결과 검증을 위한 거래ID 세션 저장

- 동일한 세션 내 요청과 결과가 동일한지 확인 및 인증결과 재사용 방지처리, 응답결과 처리 시 필수 구현
- 세션 내 거래ID를 저장하여 검증하는 방법은 권고 사항이며, 이용기관의 저장매체(DB 등)에 저장하여 검증 가능

clientTxId 세션 저장

- clientTxId 는 인증 요청과 결과 프로세스의 일치성을 검증하기 위해 세션 또는 이용기관의 저장매체(DB 등)에 저장에 저장해야 합니다.
- 예) 세션 저장

```
HttpSession session = request.getSession();
session.setAttribute("clientTxId", clientTxId);
session.setMaxInactiveInterval(10 * 60); // 10분 만료
```

- 검증 완료 후 clientTxId 는 즉시 사용 완료 처리(삭제)하여 재사용을 방지합니다.

Step 2. 본인확인 토큰요청 데이터 생성

- **Step 2-1.** 암호화 전 데이터 생성 (JSONData 생성)합니다.
- **Step 2-2.** JSON 데이터 직렬화

① JSONData 생성 및 직렬화 규칙

이용기관 거래 ID(clientTxId), 버전, 현재 요청 시간을 포함하는 JSON 객체를 생성하고, 이를 문자열로 직렬화합니다.

- version : "V2"
- clientTxId : 이용기관 거래 ID (예 : ORG001-550e8400-e29b-41d4-a716-44665544)
- requestTime : 현재 요청 시간 (형식 : yyyyMMddHHmmss, 예: 20250318113000)
- JSON 객체를 문자열로 직렬화하여 암호화에 사용할 텍스트 데이터를 준비합니다. (예: JSON 직렬화 라이브러리 사용, UTF-8 인코딩)

예시)

```
{
  "version" : "V2",
  "clientTxId" : "ORG001-550e8400-e29b-41d4-a716-44665544",
  "requestTime" : "20250318113000"
}
```

- **Step 2-3. 암호화 진행(RSA 암호화) 합니다.**
 1. RSA 서버 공개키 준비
 - 공개키를 Base64 디코딩 하여 바이트 배열(byte[]) 형태로 읽어서 준비
 2. RSA 암호화 방식 설정
 - RSA/ECB/OAEPWithSHA-256AndMGF1Padding 모드(OAEP: SHA-256, MGF1: SHA-256)로 위에서 생성한 데이터(직렬화된 데이터)를 암호화
 3. Base64 인코딩 처리
 - 암호화된 데이터를 Base64로 인코딩하여 전송 가능한 문자열 형태로 변환

⚠ Warning

RSA-OAEP 평문 길이 제한(2048-bit 기준 약 190B 전후)

i Info

- `serverPublicKeyBase64` 는 **1** **본인확인 암호화용 키 정보** 가이드에서 `mok_keyInfo.dat` 파일 복호화로 얻은 **ServerPublicKey** 값을 사용합니다. (요청 암호화용)
- 공개키는 X.509 포맷으로 관리됨

Step 3. 결과 확인

- **Step 3-1. Base64 인코딩된 본인확인 요청 토큰(encryptReqClientInfo)**

```
"WBIF9sY4qATKJD+CrXMx3Ml7 ... cf6z+a5pX+pvebiFmt6ChieUnpZp1pLA=="
```

Step 4. 표준창 인증(거래) 요청정보 생성

MOKReqClientInfo

이름	타입	설명	필수
serviceld	String	본인확인 이용기관 서비스 ID (본인확인 암호화용 키 정보 내 포함됨)	O
encryptReqClientInfo	String	암호화된 본인확인 거래 요청 정보	O
serviceType	String	이용상품 코드 "telcoAuth" : 휴대폰본인확인 서비스 "telcoAuth-LMS" : 휴대폰본인확인 LMS 서비스	O
usageCode	String	서비스 이용 코드 - 01001 : 회원가입 - 01002 : 정보변경 - 01003 : ID찾기 - 01004 : 비밀번호찾기 - 01005 : 본인확인용 - 01007 : 상품구매/결제 - 01999 : 기타	O
retTransferType	String	본인확인-표준창 결과 수신 타입 (Svr to Svr) - MOKToken : 본인확인-표준창 결과 토큰	O
returnUrl	String	결과 수신(표준창→이용기관) 엔드포인트 "https://" 포함한 URL 입력	O
encryptVersion	String	본인확인-표준창 토큰 암호화 방식 - V2	O

⚠ Warning

- returnUrl 내에 개인정보를 절대 포함시켜서는 안됩니다.
- 개인정보가 부분적으로 분리되어 저장 또는 처리되더라도, 이를 조합하여 개인을 식별할 수 있는 경우에는 해당 정보는 개인정보에 해당합니다.
- (ex. hpno1=010&hpno2=1234&hpno3=5678) [휴대폰번호/성명/주민번호 등]

MOKReqClientInfo(거래 요청 정보) JSON


```
{
  "serviceId": ${serviceId},
  "encryptReqClientInfo": ${encryptReqClientInfo},
  "serviceType": ${serviceType},
  "usageCode": ${usageCode},
  "retTransferType": "MOKToken",
  "returnUrl" : ${returnUrl},
  "encryptVersion" : "V2"
}
```

위 JSON 데이터는 실제 사용 시 문자열(JSON string) 형태로 변환하여 사용해야 합니다.

샘플은 JAVA로 제공됩니다.

```
public class MOKTokenRequestEncryption {

    public static String generateEncryptReqClientInfo(String serverPublicKeyBase64) throws Exception {

        // Step 1: 이용기관 거래 ID (clientTxId) 생성
        // Step 1-1: 고유한 거래 ID 생성 (예: ORG001 + UUID)
        String identifier = "ORG001";
        // identifier 길이에 따라 총 길이가 40자를 초과할 수 있으니 필요 시 UUID substring 길이 조정
        String uuid = UUID.randomUUID().toString().replaceAll("-", "").substring(0, 32); //UUID에서 32자리 추출
        String clientTxId = identifier + "-" + uuid; // 최소 20자, 최대 40자 내로 생성
        if (clientTxId.length() < 20 || clientTxId.length() > 40) {
            throw new IllegalStateException("clientTxId length must be between 20 and 40 characters");
        }

        // Step 2: 본인확인 토큰요청 데이터 생성
        // Step 2-1: JSON 데이터 생성
        ObjectMapper mapper = new ObjectMapper();
        Map<String, String> data = new HashMap<>();
        data.put("version", "V2");
        data.put("clientTxId", clientTxId);
        data.put("requestTime", new SimpleDateFormat("yyyyMMddHHmmss").format(new Date()));
    }
}
```

```

// Step 2-2: JSON 데이터 직렬화
String jsonData;
try {
    jsonData = mapper.writeValueAsString(data);
} catch (Exception e) {
    throw new RuntimeException("JSON 직렬화 실패", e);
}

// Step 2-3: RSA 암호화
// RSA 서버 공개키 준비
byte[] publicKeyBytes = Base64.getDecoder().decode(serverPublicKeyBase64);
X509EncodedKeySpec keySpec = new X509EncodedKeySpec(publicKeyBytes);
KeyFactory keyFactory = KeyFactory.getInstance("RSA");
PublicKey publicKey = keyFactory.generatePublic(keySpec);

// RSA-OAEP(SHA-256, MGF1-SHA-256)로 암호화
OAEPParameterSpec oaepParameterSpec = new OAEPParameterSpec("SHA-256", "MGF1", MGF1ParameterSpec.SHA256,
PSource.PSpecified.DEFAULT);
Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
rsaCipher.init(Cipher.ENCRYPT_MODE, publicKey, oaepParameterSpec);
byte[] encryptedData = rsaCipher.doFinal(jsonData.getBytes(StandardCharsets.UTF_8));

// Base64 인코딩 처리
String encryptReqClientInfo = Base64.getEncoder().encodeToString(encryptedData);

// Step 3: 결과 확인
// 세션 저장 권고사항: clientTxId를 세션 또는 DB에 저장 (여기서는 예시로 출력만)
System.out.println("Generated clientTxId (for session storage): " + clientTxId);
System.out.println("Encrypted Request Client Info: " + encryptReqClientInfo);
return encryptReqClientInfo;
}

// Step 4: 표준창 인증 요청정보 생성
private static String generateMOKReqClientInfo() throws Exception {
    // 1 본인확인 암호화호용 키 정보 파일에서 정보 가져오기 가이드에서 획득한 ServerPublicKey 사용
    String serverPublicKeyBase64 = "serverPublicKeyBase64";
    String encryptedResult = generateEncryptReqClientInfo(serverPublicKeyBase64);

    ObjectMapper mapper = new ObjectMapper();

```

```

Map<String, String> data = new HashMap<>();
// 1 본인확인 암호화호용 키 정보 파일에서 정보 가져오기 가이드에서 획득한 ServiceId 사용
data.put("serviceId", "본인확인 이용기관 서비스 ID");
data.put("encryptReqClientInfo", encryptedResult);
// 이용상품 코드
// "telcoAuth" : 휴대폰본인확인
// "telcoAuth-LMS" : 휴대폰본인확인 LMS
data.put("serviceType", "이용상품 코드");
// 서비스 이용 코드
// "01001" : 회원가입
// "01002" : 정보변경
// "01003" : ID찾기
// "01004" : 비밀번호찾기
// "01005" : 본인확인용
// "01007" : 상품구매/결제
// "01999" : 기타
data.put("usageCode", "서비스 이용 코드");
data.put("retTransferType", "MOKToken");
data.put("returnUrl", "본인확인 결과를 전달할 이용기관 URL");
data.put("encryptVersion", "V2");

return mapper.writeValueAsString(data);
}

// 테스트용 메인 메서드
public static void main(String[] args) throws Exception {
    String MOKReqClientInfo = generateMOKReqClientInfo();
    System.out.println("Final MOKReqClientInfo: " + MOKReqClientInfo);
}
}

```

3. 본인확인-표준창 스크립트

3 표준창 스크립트

HTML 연동 방법

- 본인확인-표준창 JavaScript URL
 - 운영 URL : <https://cert.mobile-ok.com/resources/js/index.js>
 - 개발 URL : <https://scert.mobile-ok.com/resources/js/index.js>

이름	타입	설명	필수
MOKReqClientInfoUrl	String	POST : 본인확인 표준창 요청정보 생성 URL (이 URL의 응답 Body가 MOKReqClientInfo 를 반환해야함)	O
browser_device	String	브라우저 종류 설정 (옵션) - "WB" : PC웹 - "MB":모바일웹 - "MWV":모바일웹 View - "HY": 모바일 Hybrid App - "NA": 모바일 Native App	O
callback_function	String	callback 실행에 대한 호출 함수명 - callback 함수 사용시 : callback 함수명 - callback 함수 미사용시 : "" 입력	O

```

<!DOCTYPE html>
<html lang="ko">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8"/>
<meta name="apple-mobile-web-app-capable" content="yes"/>
<meta name="format-detection" content="telephone=no">
<meta name="viewport" content="width=device-width,initial-scale=1,minimum-scale=1,maximum-scale=1,user-scalable=no">
<!-- 개발 URL : -->
<script src="https://scert.mobile-ok.com/resources/js/index.js"></script>
<!-- 운영 URL : <script src="https://cert.mobile-ok.com/resources/js/index.js"></script> -->

```

```

<script>
  document.addEventListener("DOMContentLoaded", function () {
    document.querySelector("#mok_popup").addEventListener("click", function () {
      // 서버가 MOKReqClientInfo JSON을 만들어주는 엔드포인트(POST)
      MOBILEOK.process("/mok/client-info", "WB", "onMokResult");
    });
  });
  // 모바일 전용서비스로 페이지 이동처리 또는 카카오 브라우저 등 새창으로 처리가 어려운, 환경 또는 브라우저에서 처리
  // document.querySelector("#mok_move").addEventListener("click", function () {
    //MOBILEOK.process("{MOKReqClientInfo}", "WB", "");
  // })

  function onMokResult(payload) {
    try {
      const res = JSON.parse(payload);
      // TODO: statusCode/resultCode/mokToken 등 스키마에 맞춰 처리
    } catch (e) {
      console.error("MOK 결과 파싱 실패", e);
    }
  }
}
</script>

```

```

<title>휴대폰본인확인_표준창</title>
</head>
<body>
<div class="container">
<div class="content">
  <div class="item">
    <textarea id="result" rows="20"></textarea>
  </div>
</div>
<div class="content">
  <button id="mok_popup">본인확인 시작(팝업)</button>
</div>
<!-- <div class="content">-->
<!-- <button id="mok_move">본인확인 시작(이동)</button>-->
<!-- </div>-->
</div>
</body>

```

</html>

4. 본인확인-표준창 검증결과 요청

4 본인확인-표준창 검증결과 요청

Summary

본인확인-표준창으로부터 결과(본인확인 결과 토큰)를 수신하여, 본인확인 서버로 본인확인 토큰에 대한 결과를 요청합니다.
본인확인 결과 토큰의 수신 엔드포인트는 거래 요청 시 `returnUrl` 로 지정한 URL입니다.

Step 1. 본인확인 결과 토큰 수신 및 검증결과 요청정보

- Step 1-1. 표준창은 `Content-Type: application/x-www-form-urlencoded` (일반적) 또는 `text/plain` 로 `data=<url-encoded-json>` 형태를 전달할 수 있습니다.

예시(수신 Raw):

```
data=%257B%2522encryptMOKKeyToken%2522%253A%2522FGUD2QG4k%252F ...
8KsyGp2522resultCode%2522%253A%2522000%2522%252C%2522resultMsg%2522%253A%2522success%2522%257D
```

샘플은 JAVA로 제공됩니다.

```
@PostMapping(path = "/mok/return", consumes = MediaType.APPLICATION_FORM_URLENCODED_VALUE)
public ResponseEntity<String> receiveStdResult(@RequestParam("data") String data) throws Exception {
    // 1) URL decode
    String decoded = URLDecoder.decode(data, StandardCharsets.UTF_8.name());

    // 2) JSON 파싱
    ObjectMapper mapper = new ObjectMapper();
    Map<String, String> map = mapper.readValue(decoded, Map.class);

    // 3) 필수 필드 확인
    String token = map.get("encryptMOKKeyToken");
```

```

if (token == null || token.isBlank()) {
    return ResponseEntity.badRequest().body("본인확인 결과 토큰(encryptMOKKeyToken)이 없습니다.");
}

// 4) 즉시 서버 내부로 토큰 전달 → 5초 TTL 내 검증 요청
//    (동일 요청의 재사용 방지를 위해 clientTxId(세션/DB)와 매핑, 처리 후 즉시 폐기)
// 비동기 큐/스레드풀 사용 시, 처리 지연으로 TTL 초과하지 않게 주의 (NTP 시간 동기 필수)

// TODO: 검증 요청 호출 (아래 Step 2 참고)

return ResponseEntity.ok("OK");
}

```

⚠ 주의

본인확인 검증 결과는 본인확인 결과 토큰 발급 시점으로부터 5초내에 (TTL 5초) 본인확인 결과 요청을 하여야 합니다.

Step 2. 본인확인 검증결과 요청

- 본인확인-표준창 검증 결과 요청 URL

메서드	URL	환경
POST	https://scert.mobile-ok.com/gui/service/v1/result/request	개발
POST	https://cert.mobile-ok.com/gui/service/v1/result/request	운영

1 요청

헤더

이름	설명	필수
Content-Type	Content-Type : application/json; charset=UTF-8 요청 데이터 타입	O

본문

이름	타입	설명	필수
encryptMOKKeyToken	String	본인확인-표준창으로부터 전달받은 본인확인 결과 토큰	O

2 응답

본문

이름	타입	설명	필수
resultCode	String	응답 결과 코드 -"2000" 이면 성공 - 그 외 실패 처리	O
resultMsg	String	실패 시 에러 설명	O
serviceld	String	본인확인 이용기관 서비스 ID	O
encryptMOKResult	String	암호화된 본인확인 검증결과 데이터	O

예제

요청

```
curl -X POST https://scert.mobile-ok.com/gui/service/v1/result/request \
-H "Content-Type: application/json; charset=UTF-8" \
-d '{
    "encryptMOKKeyToken" : ${encryptMOKKeyToken}
}'
```

응답 : 성공

```
{
  "encryptMOKResult" : "ln0XRGpb//+vzabdYpcjj0lmivt8qI ... 3eUZgz1jaqU0ZRXz8=",
  "resultCode" : "2000",
  "serviceId" : "690a6a66-2d ... 900d-43f29",
  "resultMsg" : "success"
}
```

5. 본인확인-표준창 개인정보 복호화

5 본인확인 개인정보 복호화

⚠ 중요

키파일은 서버 내 안전한 장소에 보관

서비스 등록 후 발급된 `mok_keyInfo.dat` 를 안전한 서버 내 디렉토리에 보관

휴대폰 본인확인 인증결과 encryptMOKResult 복호화

Step 1. 암호화된 결과 데이터 준비

- **Step 1-1.** 서버로부터 받은 암호화된 인증결과(encryptMOKResult)를 준비합니다.
- **Step 1-2.** encryptMOKResult를 '|'로 분리하여 각각의 데이터를 추출합니다.

i Info

- encryptMOKResult는 두 부분으로 구성되며, '|' 구분자로 나누어짐
 - 첫 번째 부분(encryptKeyIvHashData) : RSA로 암호화된 키와 IV 정보 (Base64 인코딩)
 - 두 번째 부분(encryptResultData) : AES로 암호화된 실제 인증결과 데이터 (Base64 인코딩)

Step 2. RSA 복호화로 AES 키와 IV 추출

- **Step 2-1.** RSA 개인키를 준비합니다.
 1. 개인키를 Base64 디코딩 하여 바이트 배열(byte[]) 형태로 읽어서 준비
- **Step 2-2.** RSA 복호화 설정을 구성합니다.
 1. RSA 암호화 방식 설정

📄 RSA 암호화 알고리즘 패딩 구성

- 해시 함수: SHA-256

- 마스크 함수(MGF): MGF1 (SHA-256 기반)
- PSource: default 값 사용 (PSource.PSpecified.DEFAULT에 해당하는 값)
(SHA-256, MGF1, MGF1ParameterSpec("SHA-256"), PSource.PSpecified.DEFAULT)

⚠ 이 설정은 각 언어의 RSA 암호화 함수에서 패딩 옵션으로 명시되어야 하며, OAEP 모드를 지원하지 않는 RSA 함수는 사용할 수 없습니다.

- **Step 2-3.** RSA 복호화를 수행합니다.
 1. encryptKeyIvHashData를 Base64 디코딩하여 바이트 배열로 변환
 2. RSA 복호화를 통해 평문(keyIvHashData)을 얻음
- **Step 2-4.** keyIvHashData를 '|'로 분리하여 base64KeyIv와 hashData를 추출합니다.

Info

- 개인키(ClientPrivateKey) : 키파일 內 이용기관의 암호/복호화용 키
- 개인키는 PKCS#8 포맷으로 관리됨
- 평문(keyIvHashData)은 '|'로 구분된 두 부분으로 구성
 - 첫 번째 부분: Base64로 인코딩된 keyIv (AES 키 + IV)
 - 두 번째 부분: 원문 데이터의 SHA-256 해시값 (Base64 인코딩)

Step 3. AES 키와 IV 준비

- **Step 3-1.** base64KeyIv를 Base64 디코딩하여 48바이트 배열(keyIv)로 변환합니다.
- **Step 3-2.** keyIv에서 AES 키와 IV를 추출합니다.

Info

- 첫 32바이트: AES-256 암호화 키
- 다음 16바이트: 초기화 벡터 (iv)

Step 4. AES 복호화

- **Step 4-1.** AES/CBC/PKCS5Padding 모드로 위에서 추출한 AES 키와 IV를 이용하여 데이터를 복호화합니다.

Step 5. 결과 확인 및 검증

- **Step 5-1.** 복호화된 결과를 JSON, UTF-8 문자열로 변환
 1. 복호화된 평문 데이터를 SHA-256으로 해싱.
 2. 해시 결과를 Base64로 인코딩하여 Step 2-3에서 얻은 hashData와 비교.



중요

- JSON 형태로 복호화가 되는지 검증 필수
- 무결성 검증 : 데이터 변조 여부 확인

```
{
  "siteId" : "이용기관 ID",
  "clientTxId" : "이용기관 거래 ID",
  "txId" : "본인확인 거래 ID",
  "providerId" : "서비스제공자(인증사업자) ID",
  "serviceType" : "이용 서비스 유형",
  "ci" : "사용자 CI",
  "di" : "사용자 DI",
  "userName" : "사용자 이름",
  "userPhone" : "사용자 전화번호",
  "userBirthday" : "사용자 생년월일",
  "userGender" : "사용자 성별 (1: 남자, 2: 여자)",
  "userNation" : "사용자 국적 (0: 내국인, 1: 외국인)",
  "reqAuthType" : "본인확인 인증 종류",
  "reqDate" : "본인확인 요청 시간",
  "issuer" : "본인확인 인증 서버",
  "issueDate" : "본인확인 인증 시간"
  ...
}
```

⚠️ 중요: 이용기관 서비스 기능 처리

- 개인정보 검증 및 CI 확인
 - 이용기관에서 수신한 개인정보의 검증 처리

- 이용기관에서 수신한 CI 확인 처리
- 개인정보 관리
 - 복호화된 개인정보는 DB등 서버저장 매체에 안전하게 저장하여야 하며, 본인확인 과정에서 획득한 정보로 저장
 - 개인정보를 웹 브라우저로 전송할 경우, 외부 해킹 및 유출 방지를 위해 철저한 보안 처리 필수

샘플은 JAVA로 제공됩니다.

```
public class MOKResultDecryption {

    public static String decryptMOKResult(String encryptMOKResult, byte[] privateKey) throws Exception {
        // Step 1: 암호화된 결과 데이터 준비
        // Step 1-1 & 1-2: encryptMOKResult를 '|'로 분리
        String[] parts = encryptMOKResult.split("\\|");
        if (parts.length != 2) {
            throw new IllegalArgumentException("Invalid encryptMOKResult format");
        }
        String encryptKeyIvHashData = parts[0];
        String encryptResultData = parts[1];

        // Step 2: RSA 복호화로 AES 키와 IV 추출
        // Step 2-1 & 2-2: RSA 개인키 및 설정 준비
        Cipher rsaCipher = Cipher.getInstance("RSA/ECB/OAEPWithSHA-256AndMGF1Padding");
        KeyFactory keyFactory = KeyFactory.getInstance("RSA");
        OAEPParameterSpec oaepParams = new OAEPParameterSpec("SHA-256", "MGF1", MGF1ParameterSpec.SHA256, PSource.PSpecified.DEFAULT);
        PKCS8EncodedKeySpec privateKeySpec = new PKCS8EncodedKeySpec(privateKey);
        PrivateKey rsaPrivateKey = keyFactory.generatePrivate(privateKeySpec);

        // Step 2-3: RSA 복호화
        rsaCipher.init(Cipher.DECRYPT_MODE, rsaPrivateKey, oaepParams);
        String keyIvHashData = new String(
            rsaCipher.doFinal(Base64.getDecoder().decode(encryptKeyIvHashData)),
            StandardCharsets.UTF_8
        );

        // Step 2-4: keyIvHashData 분리
        String[] keyIvHashParts = keyIvHashData.split("\\|");
        if (keyIvHashParts.length != 2) {
```

```

        throw new IllegalArgumentException("Invalid keyIvHashData format");
    }
    String base64KeyIv = keyIvHashParts[0];
    String hashData = keyIvHashParts[1];

    // Step 3: AES 키와 IV 준비
    // Step 3-1 & 3-2: keyIv 디코딩 및 키/IV 추출
    byte[] keyIv = Base64.getDecoder().decode(base64KeyIv);
    if (keyIv.length != 48) {
        throw new IllegalArgumentException("Invalid keyIv length");
    }
    byte[] key = new byte[32];
    byte[] iv = new byte[16];
    System.arraycopy(keyIv, 0, key, 0, 32); // AES 키
    System.arraycopy(keyIv, 32, iv, 0, 16); // IV

    // Step 4: AES 복호화
    // Step 4-1: AES 설정
    Cipher aesCipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
    SecretKeySpec secretKeySpec = new SecretKeySpec(key, "AES");
    IvParameterSpec ivSpec = new IvParameterSpec(iv);
    aesCipher.init(Cipher.DECRYPT_MODE, secretKeySpec, ivSpec);

    // Step 4-2: AES 복호화
    byte[] decryptedData = aesCipher.doFinal(Base64.getDecoder().decode(encryptResultData));
    String result = new String(decryptedData, StandardCharsets.UTF_8);

    // Step 5: 결과 검증
    // Step 5-1: 무결성 검증 (SHA-256 해시 비교)
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    byte[] resultHashBytes = digest.digest(result.getBytes(StandardCharsets.UTF_8));
    String computedHash = Base64.getEncoder().encodeToString(resultHashBytes);
    if (!computedHash.equals(hashData)) {
        throw new SecurityException("Hash verification failed: Data integrity compromised");
    }

    return result;
}

// 테스트용 메인 메서드
public static void main(String[] args) throws Exception {
    // 예시 입력값

```

5. 본인확인-표준창 개인정보 복호화

```
String encryptMOKResult = "base64EncryptedKeyIvHash|base64EncryptedResultData"; // 실제 값으로 대체
byte[] privateKey = Base64.getDecoder().decode("your_base64_pkcs8_private_key"); // 실제 개인키로 대체

try {
    String decryptedResult = decryptMOKResult(encryptMOKResult, privateKey);
    System.out.println("Decrypted JSON Result: " + decryptedResult);
} catch (Exception e) {
    System.err.println("Decryption failed: " + e.getMessage());
}
}
```